COMP202 Data Structures and Algorithms

Assignment 1

Instructor: Alptekin Küpçü

Due Date: March 25, 2026, 11:59 PM

Overview

This assignment will test your ability to implement and analyze fundamental data structures from scratch. You will build stacks, queues, linked lists, and binary search trees (BSTs) without using Java's built-in collections.

Academic Integrity Statement

This assignment must be completed individually. Discussion of high-level concepts is permitted, but sharing or copying code or using AI for code generation is not permitted. By submitting your work, you agree to abide by the rules in the course syllabus and the Koç University Student Code of Conduct.

To receive a non-zero grade, you must submit a signed copy of the academic integrity statement (Honor.pdf), as stated in the course syllabus. The statement may be printed, but your personal details and signature must be filled out by hand with a **blue pen**. Acceptable file formats: .pdf, .jpg, or .png.

Technical Constrains

- **No Built-in Collections**: You are strictly forbidden from importing or using java.util collections (ArrayList, LinkedList, etc.).
- **No Arrays**: Your linked-list based implementations must not use Java arrays.
- **Provided Structure**: You will be given a src folder. You must only fill in the //TODO sections in the ds package.
    - BracketChecker.java // with TODO
    - FareySequence.java // with TODO
    - TaskManager.java // with TODO
    - Main.java // used for grading and testing

Note: Any use of built-in collections or arrays, or any modification outside the //TODO sections, will result in a **grade of 0**.

Part 1: Unmatched Brackets [20 points]

Your friend's code frequently fails due to mismatched brackets. Write a Java program using your own stack implementation (push, pop, peek(top), isEmpty) to detect and automatically correct bracket errors in the code.

The code can contain any of the following brackets: []{}().

The program should do the following:

a. **Detect unmatched closing brackets** that either
   - Do not have a corresponding opening bracket before them, or
   - Close with the wrong closing bracket (e.g., [}).

b. **Detect unclosed opening brackets** (e.g., ([ ]).

c. **Correct the input string**: output the correct version of the input string where all bracket errors have been fixed.

d. Run in $O(n)$.

Sample input	Expected output
Foo(bar[i];	Foo(bar[i]);

Part 2: Farey Sequence [40 points]

Consider fractions of level **one** to be defined as a sequence $(\frac{0}{1}, \frac{1}{1})$. This sequence can be extended such that at each level n, a new fraction $\frac{a+b}{c+d}$ is inserted between two neighbor fractions $\frac{a}{c}$ and $\frac{b}{d}$ only if $c + d \leq n$.

Level n	**Sequence**
1	$\frac{0}{1}, \frac{1}{1}$
2	$\frac{0}{1}, \frac{1}{2}, \frac{1}{1}$
3	$\frac{0}{1}, \frac{1}{3}, \frac{1}{2}, \frac{2}{3}, \frac{1}{1}$
3	$\frac{0}{1}, \frac{1}{4}, \frac{1}{3}, \frac{1}{2}, \frac{2}{3}, \frac{3}{4}, \frac{1}{1}$
5	$\frac{0}{1}, \frac{1}{5}, \frac{1}{4}, \frac{1}{3}, \frac{1}{2}, \frac{2}{3}, \frac{3}{4}, \frac{4}{5}, \frac{1}{1}$

In the FareySequence.java file, use **linked-lists** to implement the following functions:

a. ***extend()***: extends the list by **one level**, inserting all the necessary fractions.

b. ***level(n)***: extend the list to a **specific level n**, adding all fractions needed to reach that level. Ensure the input level is greater than the current level.

c. ***shrink()***: reduce the list by **one level**, removing fractions that do not belong to the lower level. Ensure that the removal is feasible.
d. ***printSequence()***: display the farey sequence of fractions.

Part 3: Task Manager [40 points]

Consider task processing system that handles incoming tasks, process them in the order they arrive, and stores them for efficient retrieval. The system uses two data structures:

(1) **Queue** to hold tasks waiting to be processed.

(2) **Binary Search Tree (BST)** to store completed tasks, enabling fast search and sorting.

Rules and details:
- Tasks are handled **strictly in arrival order**, so their priority is based on when they arrive.
- Each task contains:
  - **Task ID** (an integer, used as the key in the BST)
  - **Task Name** (a string)

- You will need to implement the following functions from scratch:
  - *enqueue()*: add a task to the queue
  - *dequeue()*: remove a task from the queue
  - *isEmpty()*: check if the queue is empty
  - *insert()*: insert a completed task into the BST
  - *remove()*: remove a task from the BST
  - *inorder()*: print all completed tasks in sorted order (**inorder** traversal)
  - *addTask()*: create a new task and enqueue it
  - *processNext()*: take the next task from the queue and process it
  - *undoTask()*: move a completed task from the BST back to the queue
  - *searchBST()*: search for a specific task in the BST using task ID

System behavior:

- When a new task arrives, a task object is created and the task is added to the queue.
- The provided timer function simulates task execution. After the task is executed, it is dequeued and inserted into the BST based on TaskID.
- The system should support undo operation for a specific TaskID, where you remove the task from the BST and add it back to the queue.
- The system should allow displaying completed tasks sorted by TaskID.

Grading Breakdown

This homework is worth a total of 100 points

- Part 1: 20 points
- Part 2: 40 points
- Part 3: 40 points (10 queue and 30 BST)

Total: 100 points

Submissions

Upload the following to LearnHub:

1. BracketChecker.java
2. FareySequence.java
3. TaskManager.java
4. Your signed Academic Integrity Statement (.jpg, .png, or .pdf)

Note: Main.java is provided for you to test your code. Passing these local tests is a good start but does not guarantee a full grade. Your final implementation grade will be determined by a more comprehensive grading.